

Step 1: Set Up AWS Credentials

You need to configure your AWS credentials to interact with AWS services using Terraform. You can either set up your credentials using the AWS CLI or provide them directly in your Terraform configuration. For simplicity, let's use the AWS CLI (learn more on how to configure credentials [here](#)).

\$ aws configure

AWS Access Key ID [None]: YOUR_ACCESS_KEY_ID

AWS Secret Access Key [None]: YOUR_SECRET_ACCESS_KEY

Default region name [None]: YOUR_REGION_NAME

Default output format [None]: json

Replace YOUR_ACCESS_KEY, YOUR_SECRET_KEY, and YOUR_REGION with your AWS access key, secret key, and preferred AWS region, respectively.

Step 2: Create a Terraform Configuration File

Create a new directory for your Terraform configuration, and inside that directory, create a file named main.tf. This file will contain your Terraform configuration for creating an S3 bucket.

```
# main.tf
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-unique-bucket-name"
  tags = {
    Name      = "MyS3Bucket"
    Environment = "Production"
  }
}
```

In this example, we have defined an AWS provider specifying the region and a basic S3 bucket resource. The configuration uses "us-east-1" as the preferred AWS region and "my-unique-bucket-name" as the name for the S3 bucket.

Step 3: Initialize Terraform

Navigate to your Terraform configuration directory in the terminal and run the following command to initialize your working directory:

```
terraform init
```

This command downloads the necessary provider plugins and sets up your Terraform environment.

Step 4: Preview Changes

After initialization, run the following command to preview the changes Terraform will make to your infrastructure:

```
terraform plan
```

Review the output to ensure that Terraform will create the resources as expected. If everything looks good, you can proceed to the next step.

Step 5: Apply Changes

To create the S3 bucket, execute the following command:

```
terraform apply
```

Terraform will prompt you to confirm the execution of the plan. Type yes and press Enter. Terraform will then create the S3 bucket and output information about the resources created.

Step 6: Verify the S3 Bucket

Visit the AWS Management Console and navigate to the S3 service. You should see the newly created bucket with the name you specified in your Terraform configuration.

Congratulations! You've successfully created an AWS S3 bucket using Terraform. But this is just the beginning. Let's explore some additional configurations and best practices.

Advanced Configurations - Using Terraform with AWS S3

Versioning

Enabling versioning for your S3 bucket helps you manage and retain multiple versions of an object. To enable versioning, modify your main.tf file:

```
# main.tf
resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-unique-bucket-name"
  tags = {
    Name      = "MyS3Bucket"
    Environment = "Production"
  }
}

resource "aws_s3_bucket_versioning" "versioning_example" {
  bucket = aws_s3_bucket.my_bucket.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

Logging

You can configure S3 bucket logging to record all requests made to your bucket. Add the following configuration to enable logging:

```
# main.tf

resource "aws_s3_bucket" "example" {
  bucket = "my-tf-example-bucket"

  tags = {
    Name      = "MyS3Bucket"
    Environment = "Production"
  }
}

resource "aws_s3_bucket_acl" "example" {
  bucket = aws_s3_bucket.example.id
  acl    = "private"
}

resource "aws_s3_bucket" "log_bucket" {
  bucket = "my-tf-log-bucket"

  tags = {
    Name      = "MyLogBucket"
    Environment = "Production"
  }
}

resource "aws_s3_bucket_acl" "log_bucket_acl" {
  bucket = aws_s3_bucket.log_bucket.id
  acl    = "log-delivery-write"
}

resource "aws_s3_bucket_logging" "example" {
  bucket          = aws_s3_bucket.example.id
  target_bucket   = aws_s3_bucket.log_bucket.id
  target_prefix   = "log/"
}
```

This example creates a separate S3 bucket for logs and configures the main bucket to log to the log bucket.

Access Control

AWS Identity and Access Management (IAM) allows you to control access to your S3 bucket. You can define IAM policies and attach them to your bucket. Here's a simple example:

```
# main.tf

resource "aws_s3_bucket" "example" {
  bucket = "my-tf-test-bucket"
}

resource "aws_s3_bucket_policy" "allow_access_from_another_account" {
  bucket = aws_s3_bucket.example.id
  policy = data.aws_iam_policy_document.allow_access_from_another_account.json
}

data "aws_iam_policy_document" "allow_access_from_another_account" {
  statement {
    principals {
      type        = "AWS"
      identifiers = ["123456789012"]
    }
    actions = [
      "s3:GetObject",
      "s3:ListBucket",
    ]
    resources = [
      aws_s3_bucket.example.arn,
      "${aws_s3_bucket.example.arn}/*",
    ]
  }
}
```

This example creates an IAM user and grants full access to the S3 bucket.